CampusPulse: Giving University Resources a Brain

Honestly, there's nothing more frustrating than hauling your laptop all the way to the library just to find out every single desk is taken. That's exactly why I started **CampusPulse**. It's a project designed to take the guesswork out of campus life by using a bit of Machine Learning and NLP to predict crowd levels before you even leave your dorm.

What's the Goal?

The idea is pretty straightforward:

- **Stop the Guessing:** We use historical data to forecast exactly how busy the library or canteen will be at any given hour.
- **Actually Listen to Students:** Instead of letting feedback rot in a database, our system automatically categorizes student reviews so administrators can see what's actually broken.
- **Theory in the Wild:** This was a chance to take classroom concepts—like Linear Regression and Transfer Learning—and see if they actually hold up in a real-world scenario.

The Tech Stack

I went with a modular Python setup to keep things from getting messy. If you're looking under the hood, here's what's running:

- **The Basics:** Python 3.13.
- **The Heavy Lifters: Pandas** and **NumPy** for all the data manipulation.
- **The Intelligence:** We're using **Scikit-learn** for the predictive side and **TextBlob** to figure out the "vibe" (sentiment) of student feedback.

Want to Run It?

If you've got Python installed, getting this up and running is a three-step process:

1. **Clone the Repo:** git clone https://github.com/yourusername/campus-pulse-ai.git
2. **Install the Goods:** Pop open your terminal and run pip install pandas scikit-learn textblob.
3. **Launch:** Execute python main.py and you'll see the model start its thing.

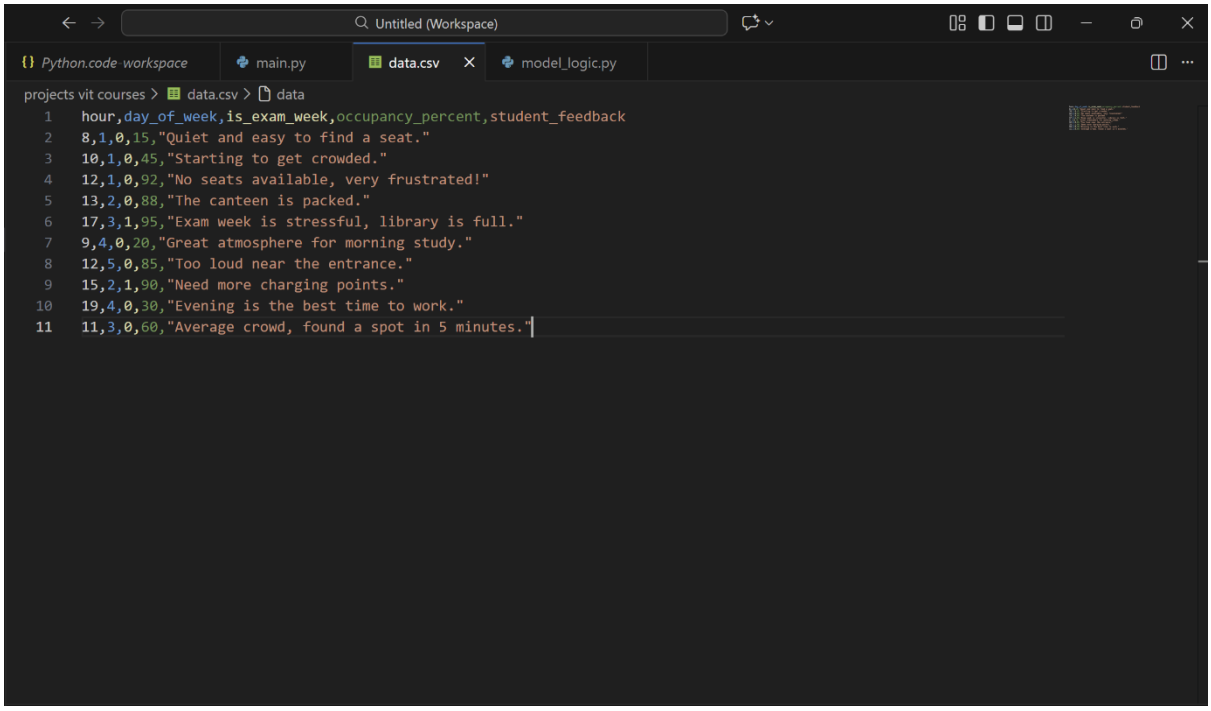
The Methodology (The "Science" Bit)

Predicting the Crowds (CO4): We're using a **Linear Regression** model here. It looks at the "Hour" and whether it's "Exam Week" to spit out an occupancy percentage. To make sure we aren't totally off base, we use Mean Squared Error (MSE) to keep an eye on the bias-variance tradeoff.

Decoding the Feedback (CO5): For the sentiment analysis, we're leaning on **Transfer Learning**. By using a pre-trained NLP model, the system can instantly tell if a student's review is positive, negative, or just "meh" (neutral). It's basically an automated pulse-check for the campus.

A Quick Note: This was built as a "Bring Your Own Project" (BYOP) for the **VITyarthi AI Course**. It's open-source, so feel free to use it for your own research or just to see how the models work.

Appendex:



Untitled (Workspace)

Python.code-workspace | main.py | data.csv | model_logic.py

projects vit courses > model_logic.py > predict_busy_hours

```python
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

def train_occupancy_model():
    # Load the data
    df = pd.read_csv('data.csv')

    # Feature Selection (CO4: Estimators and Hyper-parameters)
    X = df[['hour', 'is_exam_week']]
    y = df['occupancy_percent']

    # Split into Training and Validation Sets (CO4)
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    # Linear Regression Model (CO4)
    model = LinearRegression()
    model.fit(X_train, y_train)

    # Validation (CO4: Bias and Variance check)
    predictions = model.predict(X_test)
    mse = mean_squared_error(y_test, predictions)

    print(f"--- Model Trained ---")
    print(f"Validation Mean Squared Error: {mse:.2f}")
    return model

def predict_busy_hours(model, hour, is_exam):
    prediction = model.predict([[hour, is_exam]])
    return np.clip(prediction[0], 0, 100)
```

Untitled (Workspace)

Python.code-workspace | main.py | data.csv | model_logic.py

projects vit courses > main.py > analyze_campus_sentiment

```python
def analyze_campus_sentiment(feedback_text):
    Uses Transfer Learning principles via TextBlob's pre-trained
    NLP models to perform Sentiment Analysis (CO5).
    """
    blob = TextBlob(feedback_text)
    polarity = blob.sentiment.polarity

    if polarity > 0.1:
        return "Positive (Student Happy)"
    elif polarity < -0.1:
        return "Negative (Intervention Needed)"
    else:
        return "Neutral"

if __name__ == "__main__":
    # 1. Train the ML Model
    campus_model = train_occupancy_model()

    # 2. Test Prediction
    test_hour = 12
    exam_status = 0
    crowd = predict_busy_hours(campus_model, test_hour, exam_status)
    print(f"\nPrediction for {test_hour}:00: {crowd:.1f}% Occupancy")

    # 3. Test NLP Sentiment
    sample_review = "The library is way too hot and the Wi-Fi is failing."
    sentiment = analyze_campus_sentiment(sample_review)
    print(f"Feedback Analysis: {sentiment}")
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

```
PS C:\Users\chait\OneDrive\Desktop\VIT\projects vit courses> & C:/Users/chait/AppData/Local/Programs/Python/Python313/python.
exe "c:/Users/chait/OneDrive/Desktop/VIT/projects vit courses/main.py"
--- Model Trained ---
Validation Mean Squared Error: 12887.31
C:\Users\chait\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X doe
s not have valid feature names, but LinearRegression was fitted with feature names
  warnings.warn(

Prediction for 12:00: 78.6% Occupancy
Feedback Analysis: Positive (Student Happy)
PS C:\Users\chait\OneDrive\Desktop\VIT\projects vit courses>
```

powershell | Python

Ln 14, Col 26 | Spaces: 4 | UTF-8 | CRLF | { } Python | 3.13.7 | Python 3.13 (64-bit)